Compte rendu — TP Hadoop sous Ubuntu

Clusters, HDFS, YARN, MapReduce et Python

Nom / binôme : _____ Date : 01 / 11 / 2026

Environnement : VM Ubuntu (ubuntu-srv), OpenJDK 11, Hadoop 3.3.6 en mode pseudo-distribué, utilisateur etudiant (chemins /home/etudiant).

1. Captures d'écran

1.1 Installation

Java 11 installé :

```
etudiant@ubuntu-srv:~$ java --version
openjdk 11.0.32.1 2026-08-18
OpenJDK Runtime Environment (build 11.0.32.1+1-post-1ubuntu1-26.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 11.0.32.1+1-post-1ubuntu1-26.04-Ubuntu, mixed mode,
sharing)
```

Capture : java --version

Hadoop 3.3.6 reconnu (variables d'environnement correctes) :

```
tudiant@ubuntu-srv:~$ hadoop version
adoop 3.3.6
```

Capture : hadoop version

Formatage du NameNode réussi :

```
026-11-01 07:15:40,306 INFO common.Storage: Storage directory /home/etudiant/hadoop
data/namenode has been successfully formatted.
```

Capture : hdfs namenode -format

1.2 Sortie de jps

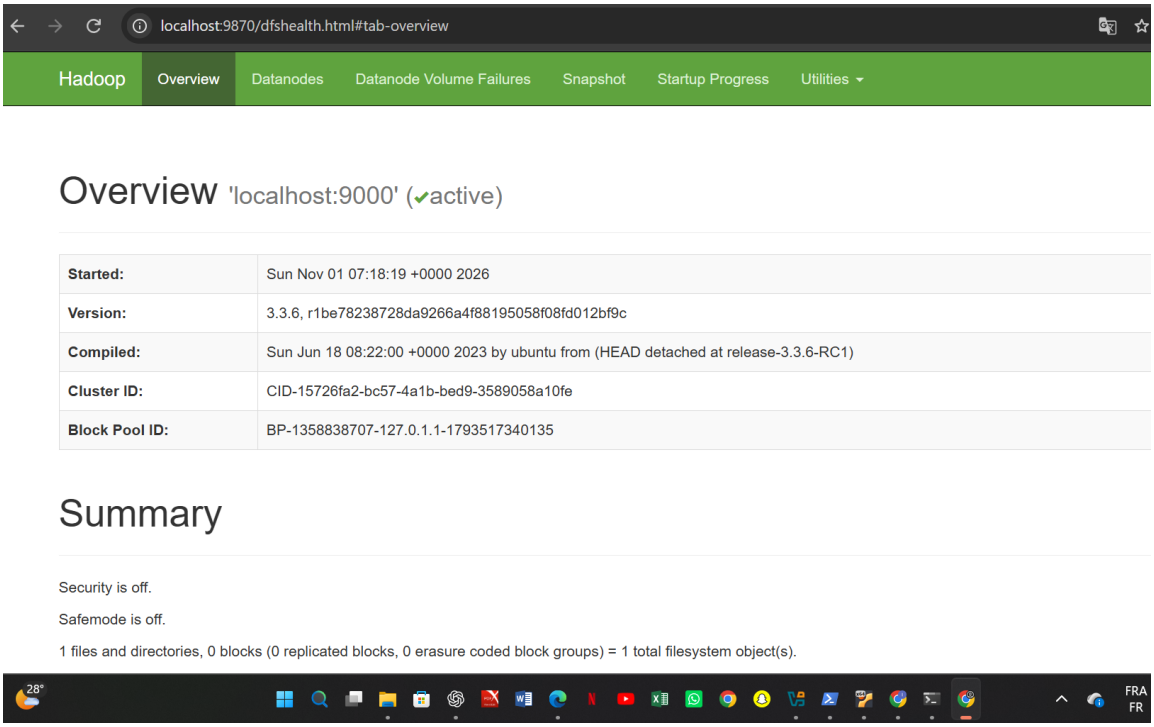
Les 5 démons Hadoop sont actifs (NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager).

```
etudiant@ubuntu-srv:~/hadoop/etc/hadoop$ jps
3427 Jps
2659 SecondaryNameNode
2277 NameNode
2407 DataNode
2875 ResourceManager
3004 NodeManager
```

Capture : jps

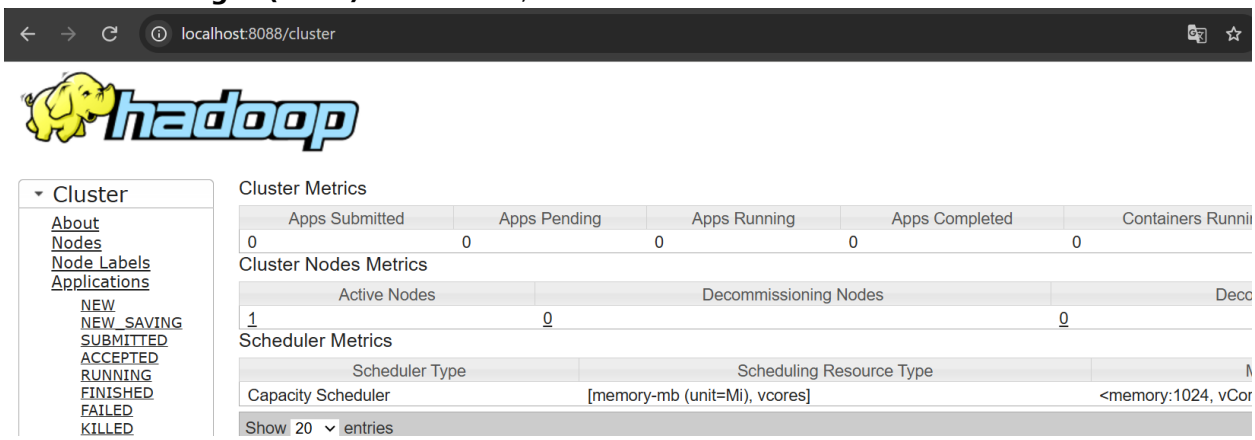
1.3 Interfaces web

NameNode (9870) : HDFS actif sur localhost:9000, version 3.3.6, Safemode off.



Capture : http://localhost:9870

ResourceManager (8088) : YARN actif, 1 nœud actif.



Capture : http://localhost:8088

1.4 Résultat de chaque job

Exercice 1 — HDFS : poeme.txt (85 octets) occupe 1 bloc, statut HEALTHY.

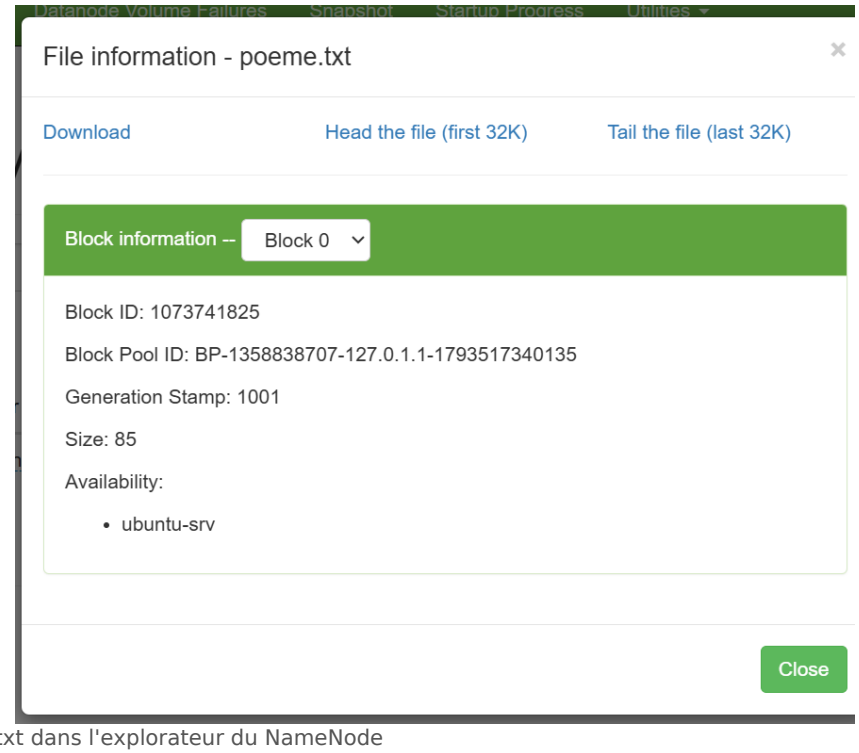
```
udiant@ubuntu-srv:~/tp$ hdfs dfs -put poeme.txt /user/hadoop/tp/input/
fs dfs -ls /user/hadoop/tp/input
fs dfs -cat /user/hadoop/tp/input/poeme.txt
fs fsck /user/hadoop/tp/input/poeme.txt -files -blocks -locations
und 1 items
w-r--r--   1 etudiant supergroup         85 2026-11-01 07:52 /user/hadoop/tp/input
oeme.txt
 chat dort sur le tapis
 chien court dans le jardin
 chat court apres le chien
nnecting to namenode via http://localhost:9870/fsck?ugi=etudiant&files=1&blocks=1&
cations=1&path=%2Fuser%2Fhadoop%2Ftp%2Finput%2Fpoeme.txt
CK started by etudiant (auth:SIMPLE) from /127.0.0.1 for path /user/hadoop/tp/inpu
poeme.txt at Sun Nov 01 07:53:16 UTC 2026

ser/hadoop/tp/input/poeme.txt 85 bytes, replicated: replication=1, 1 block(s):  OK
 BP-1358838707-127.0.1.1-1793517340135:blk_1073741825_1001 len=85 Live_repl=1  [Da
nodeInfoWithStorage[127.0.0.1:9866,DS-2f4f717c-d745-493f-8d58-2d8413c4e86a,DISK]]


atus: HEALTHY
umber of data-nodes:  1
umber of racks:               1
otal dirs:                    0
otal symlinks:                0

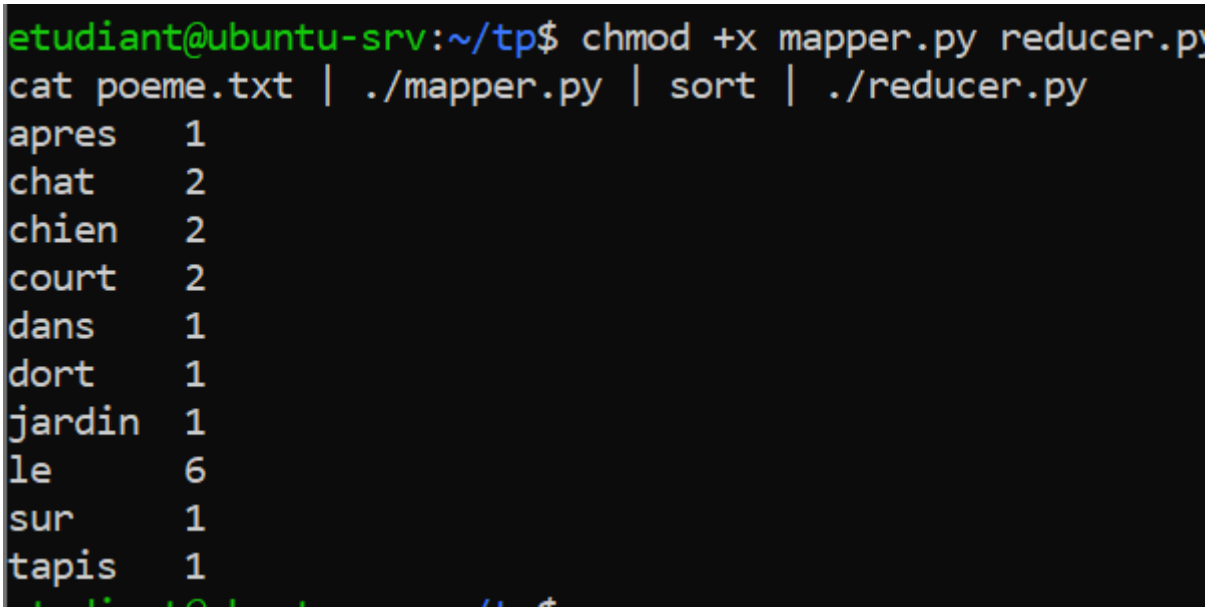
plicated Blocks:
otal size:    85 B
otal files:   1
otal blocks (validated):      1 (avg. block size 85 B)
inimally replicated blocks:   1 (100.0 %)
ver-replicated blocks:        0 (0.0 %)
nder-replicated blocks:       0 (0.0 %)
is-replicated blocks:         0 (0.0 %)
efault replication factor:    1
verage block replication:     1.0
issing blocks:                0
orrupt blocks:                0
issing replicas:              0 (0.0 %)
```

Capture : hdfs fsck poeme.txt -files -blocks -locations

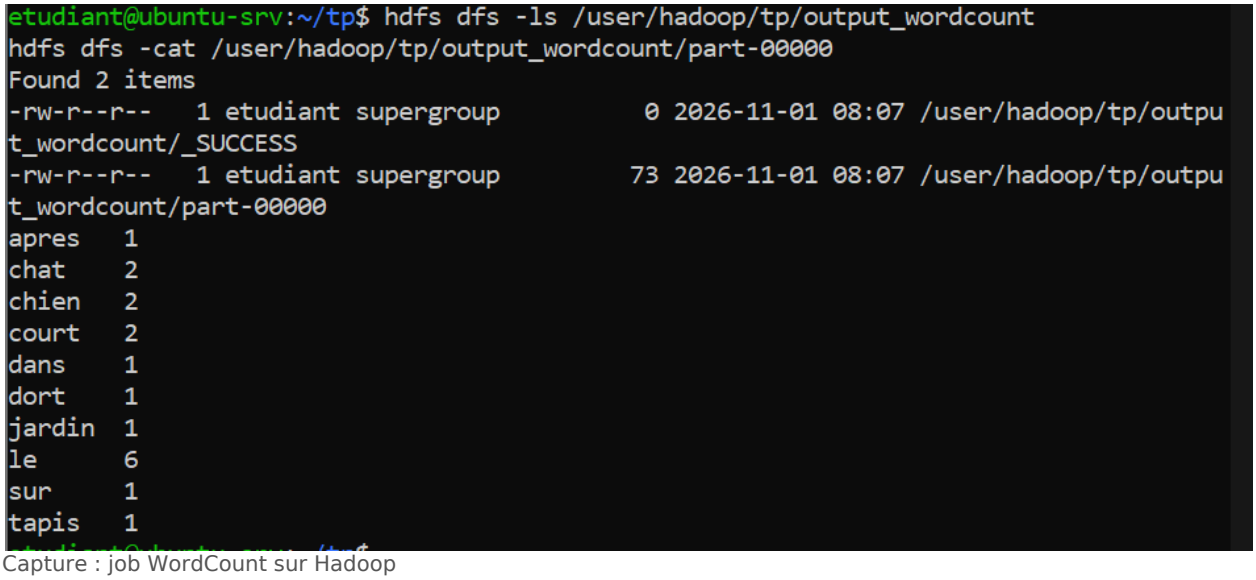


Capture : poeme.txt dans l'explorateur du NameNode

Exercice 2 — WordCount : test local, puis job Hadoop (_SUCCESS et part-00000) avec le même résultat.



Capture : test local cat | mapper | sort | reducer



Capture : job WordCount sur Hadoop

Test sur un vrai texte : les 20 mots les plus fréquents du livre (surtout des mots outils : de, il, la, et, le).

```
etudiant@ubuntu-srv:~/tp$ hdfs dfs -cat /user/hadoop/tp/output_livre/part-* | sort
k2 -n -r | head -20
de      4472
il      3192
la      3040
et      2949
le      2548
l       2391
à       2324
un      1812
les     1537
que     1350
une     1314
d       1293
qui     1278
qu      1210
en      1194
était   1175
dans    1134
est     1130
ce      1062
des     948
```

Capture : 20 mots les plus fréquents

Exercice 3 — Chiffre d'affaires par ville : résultat conforme à l'attendu.

```
output_ventes
Abidjan 910000  3
Bouake  445000  2
Korhogo 40000   1
San-Pedro       260000  2
Yamoussoukro    540000  2
etudiant@ubuntu-srv:~/tp$ _
```

Capture : résultat du job ventes

Exercice 4 — Température maximale par année : 2024 : 35.0 et 2025 : 36.1.

```
2024    35.0
2025    36.1
```

Capture : résultat du job météo

Exercice 5 — Moyenne par matière : job non exécuté, le TP ne fournit pas de fichier notes.csv.
Exercice 6 — Index inversé : job non exécuté sur Hadoop ; résultat attendu avec les fichiers d'exemple :

Fichiers d'entrée (exemple)

```
doc1.txt : le chat dort sur le tapis
doc2.txt : le chien court dans le jardin
doc3.txt : le chat court dans la maison
```

Résultat attendu

```
chat      doc1.txt,doc3.txt
chien     doc2.txt
court     doc2.txt,doc3.txt
dans      doc2.txt,doc3.txt
dort      doc1.txt
jardin    doc2.txt
la        doc3.txt
le        doc1.txt,doc2.txt,doc3.txt
maison    doc3.txt
sur       doc1.txt
```

```
tapis     doc1.txt
```

2. Code source commenté (exercices 2 à 6)

Exercice 2 — WordCount

Le **mapper** découpe le texte en mots et émet (mot, 1). Hadoop trie par mot, puis le **reducer** additionne les 1 de chaque mot.

mapper.py

```
#!/usr/bin/env python3
import sys, re

for ligne in sys.stdin:                    # lit le fichier ligne par ligne (stdin)
    ligne = ligne.strip().lower()          # supprime les espaces, met en minuscules
    mots = re.findall(r"[a-zàâäéèêëîïôöùûüç]+", ligne)   # extrait les mots
    for mot in mots:
        print(f"{mot}\t1")                 # émet la paire : mot <TAB> 1
```

reducer.py

```
#!/usr/bin/env python3
import sys

mot_courant, total = None, 0               # mot en cours et son compteur

for ligne in sys.stdin:                    # lignes déjà triées par mot
    mot, valeur = ligne.rstrip("\n").split("\t", 1)
    try:
        valeur = int(valeur)
    except ValueError:
        continue                           # ignore une ligne mal formée
    if mot == mot_courant:
        total += valeur                    # même mot : on additionne
    else:
        if mot_courant is not None:
            print(f"{mot_courant}\t{total}")   # mot terminé : on l'écrit
        mot_courant, total = mot, valeur       # nouveau mot
if mot_courant is not None:
    print(f"{mot_courant}\t{total}")           # dernier mot
```

Commande Hadoop Streaming

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
  -files mapper.py,reducer.py \
  -mapper "python3 mapper.py" -reducer "python3 reducer.py" \
  -input /user/hadoop/tp/input -output /user/hadoop/tp/output_wordcount
```

Exercice 3 — Chiffre d'affaires par ville

Le **mapper** calcule quantité x prix et émet (ville, montant). Le **reducer** cumule le total et compte les ventes par ville.

mapper_ventes.py

```
#!/usr/bin/env python3
import sys

for ligne in sys.stdin:
    ligne = ligne.strip()
    if not ligne or ligne.startswith("date"):   # ignore ligne vide et en-tête
        continue
    champs = ligne.split(",")
    if len(champs) != 5:                        # ignore une ligne incomplète
        continue
    _, ville, _, quantite, prix = champs
    try:
        montant = int(quantite) * float(prix)   # montant = quantité x prix
    except ValueError:
        continue
    print(f"{ville}\t{montant}")                # clé = ville, valeur = montant
```

reducer_ventes.py

```
#!/usr/bin/env python3
import sys
```

```
ville_courante, total, nb = None, 0.0, 0        # ville en cours, total, nb de ventes

def emettre(ville, total, nb):
    print(f"{ville}\t{total:.0f}\t{nb}")         # ville, chiffre d'affaires, nb ventes

for ligne in sys.stdin:                          # lignes triées par ville
    ville, montant = ligne.rstrip("\n").split("\t")
    montant = float(montant)
    if ville == ville_courante:
        total += montant                         # même ville : on cumule
        nb += 1
    else:
        if ville_courante is not None:
            emettre(ville_courante, total, nb)  # ville terminée
        ville_courante, total, nb = ville, montant, 1
if ville_courante is not None:
    emettre(ville_courante, total, nb)          # dernière ville
```

Exercice 4 — Température maximale par année

Le **mapper** émet (année, température). Le **reducer** garde le maximum au lieu de faire une somme.

mapper_meteo.py

```
#!/usr/bin/env python3
import sys

for ligne in sys.stdin:
    champs = ligne.strip().split(",")
    if len(champs) != 3:                    # ignore une ligne mal formée
        continue
    annee, _, temp = champs
    try:
        float(temp)                         # vérifie que c'est un nombre
    except ValueError:
        continue
    print(f"{annee}\t{temp}")               # clé = année, valeur = température
```

reducer_meteo.py

```
#!/usr/bin/env python3
import sys

cur, mx = None, None                        # année en cours et son maximum

for ligne in sys.stdin:                     # lignes triées par année
    annee, temp = ligne.rstrip("\n").split("\t")
    temp = float(temp)
    if annee == cur:
        mx = max(mx, temp)                  # on garde le maximum (pas de somme)
    else:
        if cur is not None:
            print(f"{cur}\t{mx}")          # année terminée
        cur, mx = annee, temp
if cur is not None:
    print(f"{cur}\t{mx}")                   # dernière année
```

Exercice 5 — Moyenne des notes par matière

Le **mapper** émet (matière, note, 1). Le **combiner** additionne sommes et effectifs sans diviser ; le **reducer** divise à la fin.

mapper_notes.py

```
#!/usr/bin/env python3
import sys

for ligne in sys.stdin:
    ligne = ligne.strip()
    if not ligne or ligne.startswith("etudiant"):   # ignore l'en-tête
        continue
    champs = ligne.split(",")
    if len(champs) != 3:
        continue
    _, matiere, note = champs
    try:
```

```
        float(note)
    except ValueError:
        continue
    print(f"{matiere}\t{note}\t1")         # matière, note, effectif 1
```

combiner_notes.py

```
#!/usr/bin/env python3
import sys

cur, s, c = None, 0.0, 0                   # matière, somme des notes, effectif

for ligne in sys.stdin:                    # lignes triées par matière
    m, n, k = ligne.rstrip("\n").split("\t")
    n, k = float(n), int(k)
    if m == cur:
        s += n                             # additionne les notes
        c += k                             # additionne les effectifs
    else:
        if cur is not None:
            print(f"{cur}\t{s}\t{c}")      # sortie partielle, sans diviser
        cur, s, c = m, n, k
if cur is not None:
    print(f"{cur}\t{s}\t{c}")
```

reducer_notes.py

```
#!/usr/bin/env python3
import sys

cur, s, c = None, 0.0, 0                   # identique au combiner...

for ligne in sys.stdin:
    m, n, k = ligne.rstrip("\n").split("\t")
    n, k = float(n), int(k)
    if m == cur:
        s += n
        c += k
    else:
        if cur is not None:
            print(f"{cur}\t{s / c:.2f}")   # ... mais on divise ici : moyenne
        cur, s, c = m, n, k
if cur is not None:
    print(f"{cur}\t{s / c:.2f}")
```

Exercice 6 — Index inversé

Le **mapper** émet (mot, nom du fichier). Le **reducer** regroupe les fichiers distincts qui contiennent chaque mot.

mapper_index.py

```
#!/usr/bin/env python3
import sys, os, re

# nom du fichier en cours de lecture (variable d'environnement de Hadoop)
chemin = os.environ.get("mapreduce_map_input_file") or os.environ.get("map_input_file", "inconnu")
fichier = os.path.basename(chemin)

for ligne in sys.stdin:
    for mot in re.findall(r"[a-zàâäéèêëîïôöùûüç]+", ligne.lower()):
        print(f"{mot}\t{fichier}")         # clé = mot, valeur = nom du fichier
```

reducer_index.py

```
#!/usr/bin/env python3
import sys

cur, fichiers = None, set()                # mot en cours, ensemble de fichiers (sans doublon)

for ligne in sys.stdin:                    # lignes triées par mot
    mot, f = ligne.rstrip("\n").split("\t")
    if mot == cur:
        fichiers.add(f)                    # même mot : on ajoute le fichier
    else:
        if cur is not None:
            print(f"{cur}\t{','.join(sorted(fichiers))}")   # mot terminé
        cur, fichiers = mot, {f}
```

```
if cur is not None:
    print(f"{cur}\t{','.join(sorted(fichiers))}")
```

3. Réponses aux questions

3.1 Questions des encadrés

Exercice 1 — HDFS

1) Combien de blocs poeme.txt occupe-t-il ? Pourquoi ?

1 bloc : le fichier (85 octets) est bien plus petit que la taille d'un bloc (128 Mo). Il n'occupe que sa taille réelle.

2) Retrouver le fichier dans l'interface du NameNode.

Fait : http://localhost:9870, Utilities, Browse the file system, /user/hadoop/tp/input/poeme.txt (voir capture en 1.4).

3) Nombre de blocs d'un fichier de 1 Go, et espace avec réplication 3 ?

1024 Mo / 128 Mo = 8 blocs. Avec réplication 3 : 24 blocs, soit 3 Go d'espace disque.

Exercice 5 — Moyenne par matière

Peut-on utiliser le reducer de moyenne comme combiner ? Justifier.

Non : la moyenne des moyennes n'est pas la moyenne globale si les effectifs diffèrent. Le mapper émet matière, note, 1 ; combiner et reducer additionnent sommes et effectifs, et on divise seulement à la fin.

3.2 Questions de synthèse

1. Définition d'un cluster et trois avantages.

Ensemble d'ordinateurs (nœuds) reliés en réseau et coordonnés par un logiciel, fonctionnant comme un seul système. Avantages : gros volumes, calcul parallèle, haute disponibilité.

2. Scalabilité verticale et horizontale.

Verticale : machine plus puissante (limitée, chère). Horizontale : ajout de machines. Hadoop exploite l'horizontale.

3. Rôle du NameNode et point sensible.

Il gère l'arborescence HDFS et la localisation des blocs (sans stocker de données). S'il tombe, HDFS est inaccessible.

4. Le Secondary NameNode est-il une copie de secours ?

Non. Il fusionne périodiquement edits et fsimage (checkpoint) ; il ne remplace pas le NameNode en cas de panne.

5. Localité des données.

Envoyer le programme (petit) vers les données (volumineuses). Cela réduit le trafic réseau et accélère le traitement.

6. Phases d'un job MapReduce.

Input/Split, Map, Combine (facultatif), Partition, Shuffle & Sort, Reduce, Output. Le développeur écrit Map, Combine et Reduce.

7. Pourquoi les lignes d'une même clé sont-elles consécutives ?

Parce que Hadoop trie les paires par clé (Shuffle & Sort) avant de les donner au reducer.

8. Quand utiliser un combiner ?

Quand l'opération est associative et commutative (somme, max). Contre-exemple : la moyenne (il faut transporter somme et effectif).

9. Panne d'un nœud pendant une tâche Map.

L'ApplicationMaster relance la tâche sur un autre nœud, grâce aux répliques HDFS du bloc. Le job continue.

10. Deux limites de MapReduce et un outil.
Lent pour les traitements itératifs (écritures disque) et latence élevée (pas de temps réel). Spark répond à ces limites.